codex-windows / 019f3635-106a-7d60-b364-5010020f3216

Metadata

- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-06T12-24-40-019f3635-106a-7d60-b364-5010020f3216.jsonl`
- SHA-256: `488ac8eb962e38ffaad7c3de886563873624c0612d08793af25e07248f92f622`
- Source modified: `2026-07-06T07:41:33+00:00`
- Imported at: `2026-07-08T15:59:09+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\Agent Sessions`
- model_provider: `openai`
- session_id: `019f3635-106a-7d60-b364-5010020f3216`
- source: `vscode`

Transcript

1. developer (2026-07-06T06:55:10.580Z)

<permissions instructions>
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled. Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason, commands will be rejected.
</permissions instructions>
<app-context>
Codex desktop context

- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files

- In the app, the model can display images and videos using standard Markdown image syntax: ![alt](url)
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., ![alt](/absolute/path.png)); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., [label](https://example.com)).

Workspace Dependencies

- For sheets, slides, and documents, call `load_workspace_dependencies` to find the bundled runtime and libraries.

Automations

- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the `automation_update` tool first, then follow its schema instead of writing raw automation directives by hand.
- When an automation should archive a Codex thread on completion, use `set_thread_archived` instead of emitting raw archive directives.

Thread Coordination

- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: `create_thread`, `fork_thread`, `list_threads`, `read_thread`, `send_message_to_thread`, `handoff_thread`,

`set_thread_pinned`, `set_thread_archived`, or `set_thread_title`.
- Only use `create_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments
- Use the ::code-comment{...} directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: ::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}

Git
- Branch prefix: `codex/`. Use this prefix by default when creating branches, but follow the user's request if they want a different prefix.
- After successfully staging files, emit `::git-stage{cwd="/absolute/path"}` on its own line in your final response.
- After successfully creating a commit, emit `::git-commit{cwd="/absolute/path"}` on its own line in your final response.
- After successfully creating or switching the thread onto a branch, emit `::git-create-branch{cwd="/absolute/path" branch="branch-name"}` on its own line in your final response.
- After successfully pushing the current branch, emit `::git-push{cwd="/absolute/path" branch="branch-name"}` on its own line in your final response.
- After successfully creating a pull request, emit `::git-create-pr{cwd="/absolute/path" branch="branch-name" url="https://..." isDraft=true}` on its own line in your final response. Include `isDraft=false` for ready PRs.
- Only emit these git directives in your final response after the action actually succeeds, never in commentary updates. Keep attributes single-line.
</app-context>
<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request_user_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.
</collaboration_mode>
<apps_instructions>
Apps (Connectors)
Apps (Connectors) can be explicitly triggered in user messages in the format `[$app-name](app://{connector_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.
An app is equivalent to a set of MCP tools within the `codex_apps` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool_search` tool. If `tool_search` is available, the apps that are searchable by `tools_search` will be listed by it.
Do not additionally call list_mcp_resources or list_mcp_resource_templates for apps.
</apps_instructions>
<skills_instructions>

Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

Skill roots

- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`
- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.5/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.3/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.3/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.7/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/slack/0.1.2/skills`
- `r13` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the out (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex d (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into $CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting ima (file: r2/pdf/SKILL.md)
- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input,

output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, mod (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)
- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggering a (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from sc (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.101652/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.101652/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.101652/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on `.docx`, Word, and Google Docs-

targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final doc (file: r13/documents/26.630.12135/skills/documents/SKILL.md)
- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whe (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs

reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)
- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or canc (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and referenc (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r13/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r13/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)
- skill-creator: Create new skills, modify and improve existing skills. Use when users want to

create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-creator/SKILL.md)
- slack:slack: Read Slack context, route to the right Slack workflow, and prepare or perform Slack writes that match the user's intent. (file: r12/slack/SKILL.md)
- slack:slack-channel-summarization: Summarize activity from one Slack channel and return a concise recap, post-ready update, or summary doc. (file: r12/slack-channel-summarization/SKILL.md)
- slack:slack-daily-digest: Create a daily Slack digest from selected channels or topics. Use when the user asks for a daily Slack recap or summary of today's Slack activity. (file: r12/slack-daily-digest/SKILL.md)
- slack:slack-notification-triage: Triage recent Slack activity into a priority queue or task list for the user. (file: r12/slack-notification-triage/SKILL.md)
- slack:slack-outgoing-message: Primary skill for composing, drafting, or refining any outbound Slack content. Use this whenever the task will require using `slack_send_message`, `slack_send_message_draft`, or `slack_create_canvas`. Use `slack` to read or analyze Slack context; use this skill to produce the final outgoing message. (file: r12/slack-outgoing-message/SKILL.md)
- slack:slack-reply-drafting: Draft Slack replies from available context. Use when the user wants help finding messages that likely need a response and preparing reply drafts. (file: r12/slack-reply-drafting/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r13/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes $template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed arti (file: r13/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
  1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
  2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
  3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
  4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
  5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
  - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
  - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
  - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
  - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
  - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear

instructions), state the issue, pick the next-best approach, and continue.
</skills_instructions>
<plugins_instructions>
Plugins
A plugin is a local bundle of skills, MCP servers, and apps.
How to use plugins
- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp__server__tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
- Missing/blocked: If the user requests a plugin that does not have relevant callable capabilities for the task, say so briefly and continue with the best fallback.
</plugins_instructions>

2. user (2026-07-06T06:55:10.580Z)

AGENTS.md instructions

<INSTRUCTIONS>
Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading
Always `git pull` a repo before starting to read it ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- **Why:** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- **How (safe form):** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- **If it can't fast-forward** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts
Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit **concurrently**, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a **squash-merge silently absorbed the sibling's work**. Protocol ? do this EVERY time, not only when you suspect a collision:
- **Branch from the REMOTE, explicitly:** `git fetch origin` then `git checkout -b <name> origin/<base>`
  (e.g. `origin/master`). Never assume the current branch is the intended base.
- **Re-verify right before `git add`/commit:** `git branch --show-current` + `git log --oneline -1`
  (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your
  commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- **Stage explicit paths** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked
  files can't ride into your commit.
- **Serialize repo writes:** don't start a repo-writing spawned/background task while you hold
  uncommitted work ? commit or push first. If one may be running, treat the tree + current

branch
  as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living **session handoff** for each project and use it to resume context quickly
across windows.

- **Where it lives:**
  - If the project has an auto-memory dir (`~/.Codex/projects/<project>/memory/`), keep the
    handoff as `session-handoff.md` there, and list it under a **"? Start here"** entry at the
    top of that project's `MEMORY.md`.
  - Otherwise, keep a `SESSION_HANDOFF.md` at the repo root.
- **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates
them):
  1. **You are here** ? current state.
  2. **Next steps / open threads.**
  3. **Ramp-up kit** ? the exact files/docs to read to get current.
  4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use
it
  (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR
is
  pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a
  restart can ramp up without losing context. Do **not** rewrite it every turn ? only when
  something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the
  handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's
  attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally
verified the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run_all_tests.py` / `pytest`), not
just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let
modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-
video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI.
Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's
floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a *verified* stamp, never a rubber stamp. State plainly what was and wasn't verifiable
locally.
</INSTRUCTIONS>
<environment_context>
  <cwd>C:\Users\avidu\Projects\Agent Sessions</cwd>
  <shell>powershell</shell>
  <current_date>2026-07-06</current_date>
  <timezone>Asia/Calcutta</timezone>
  <filesystem><workspace_roots><root>C:\Users\avidu\Projects\Agent
Sessions</root></workspace_roots><permission_profile type="disabled"><file_system
type="unrestricted" /></permission_profile></filesystem>
</environment_context>

3. user (2026-07-06T06:55:10.617Z)

Hello - This session has a starter project for you to review [avidullu/agent-
sessions#24](https://github.com/avidullu/agent-sessions/pull/24) and then once its merged. Pick
up this tracker and complete all the tasks and archive the project.

4. assistant (2026-07-06T06:55:26.912Z)

I?ll start by refreshing the checkout as instructed, then I?ll use the GitHub + code-review workflow for the PR before touching the tracker. After that I?ll inspect the project handoff so I?m not rebuilding context from scratch.

5. assistant (2026-07-06T06:55:43.808Z)

The checkout fast-forwarded cleanly and is already up to date. I?ve loaded the GitHub and code-review skill instructions; next I?m locating the handoff/tracker and the PR branch so the review is grounded in the repo?s actual current state.

6. assistant (2026-07-06T06:56:00.745Z)

A small wrinkle: the main checkout has an existing modified `SESSION_HANDOFF.md`, so I?m going to leave that worktree alone for review execution and use a separate worktree for PR #24. That keeps any sibling/session WIP from riding along with tests or branch changes.

7. assistant (2026-07-06T06:56:18.119Z)

The handoff says no open PRs, but GitHub now shows PR #24 open and mergeable, adding `docs/TECH_DEBT_PLAN.md` plus a docs index entry. I?m going to verify the doc against the code and run the repo gates from a PR worktree; the local handoff looks stale relative to GitHub and I?ll update it later only after the real state changes.

8. assistant (2026-07-06T06:56:23.193Z)

There are a couple of older worktrees, but nothing for this review branch. I?ve fetched origin and am adding a dedicated PR #24 worktree now.

9. assistant (2026-07-06T06:56:37.460Z)

The review worktree is on a local `codex/review-pr-24` branch tracking the PR head. Per the repo convention, I?m doing the fast-forward pull there before reading or running anything inside it.

10. assistant (2026-07-06T06:56:49.528Z)

Now I?m reading the new plan alongside the repo?s actual config and the code locations it cites. For this PR, the interesting question is less ?does Markdown render? and more ?is the tracker accurate enough to become the next project.?

11. assistant (2026-07-06T06:57:03.863Z)

The PR follows the tracked-project-doc template and the added docs index row is straightforward. I?m checking the cited line-level claims now, because if this doc becomes the source of truth for all follow-up work, a wrong target there would waste a lot of motion.

12. assistant (2026-07-06T06:57:20.411Z)

So far, the major cited problem areas are real in the code: the promote rewrite path drops unmarked prose, the ledger rewrite is in-place, the archive stem uses local time, and the CLI baseline dispatch range is exactly where the doc says it is. I?m moving from source inspection to the verification gates now.

13. assistant (2026-07-06T06:57:33.577Z)

I?ll verify the documented setup separately from the test environment: first I?ll try the repo?s `pip install -e ".[dev]"` in a throwaway venv, then install the missing tools directly so I can still run pytest, coverage, ruff, and mypy. That lets the review distinguish ?setup is broken? from ?the suite itself is broken.?

14. assistant (2026-07-06T06:58:18.295Z)

The editable install failure reproduces exactly as the PR says, including setuptools? flat-layout multi-package discovery error. I?m keeping that failed setup result as review evidence, then using the same venv with direct tool installs so the rest of the gates can still be checked.

15. assistant (2026-07-06T06:58:24.936Z)

README?s advertised gates are exactly `pytest --cov`, `ruff check .`, and `mypy agent_sessions tools`; there?s no separate ruff/mypy config. I?m installing those tools plus `reportlab` in the throwaway venv so optional PDF tests don?t skip.

16. assistant (2026-07-06T06:59:09.100Z)

Dependencies are in place. I?m running the review gates in parallel now: full pytest with coverage, default ruff, and the README mypy command.

17. assistant (2026-07-06T06:59:51.059Z)

The core test suite is green with coverage well above the repo floor, and default ruff is clean. The README mypy command fails on the missing reportlab stubs, matching the PR?s claim; I?m going one step further by installing the missing stubs to see whether there are deeper type errors hiding underneath.

18. assistant (2026-07-06T07:00:10.405Z)

The broader ruff audit also matches the PR exactly: 360 findings under the expanded rule set, mostly E501. I?m doing one last review-gate check against `origin/main` coverage so I can say whether this docs-only PR regresses anything, rather than just assuming it.

19. assistant (2026-07-06T07:00:17.580Z)

PR #24 has no comments, no reviews, and no configured status checks on GitHub; it?s mergeable against `origin/main` at `5d81234`. I?m creating a clean base worktree at that remote commit for the coverage comparison.

20. assistant (2026-07-06T07:00:30.176Z)

Base worktree is created from `origin/main`; I?m doing its fast-forward pull now, then I?ll run the same coverage command there.

21. assistant (2026-07-06T07:00:34.799Z)

The base worktree is clean and current. Running base coverage now; because the PR only changes docs, I expect the numbers to line up, but I?m letting the suite prove it.

22. assistant (2026-07-06T07:01:13.572Z)

Coverage on the PR and base is identical at 95.38% with `reportlab` installed, so there?s no regression. I?ve got one documentation accuracy issue left to decide on: the new tracked doc says the handoff points to it, but the PR doesn?t update `SESSION_HANDOFF.md`.

23. assistant (2026-07-06T07:01:19.007Z)

I?ve loaded the GitHub review tools too, so if I leave a review it can be attached to the PR rather than just buried in this thread. First I?m checking the connector?s view of comments/threads to make sure I?m not missing existing review context.

24. assistant (2026-07-06T07:01:34.060Z)

The GitHub connector doesn?t have access to this private repo, but `gh` does and already showed no comments or review threads. I found a small but real doc-consistency issue, and since it blocks a clean approval more than the underlying plan, I?m going to patch the PR branch rather than bounce it back.

25. assistant (2026-07-06T07:01:56.015Z)

I?m updating only the handoff text needed to make the new tracker discoverable after merge: add `docs/TECH_DEBT_PLAN.md` as the next tracked project and include it in the ramp-up kit. That resolves the review issue without changing the tracker itself.

26. assistant (2026-07-06T07:02:14.299Z)

The handoff patch is tiny and docs-only. I?m verifying the branch/HEAD before staging, then I?ll commit and push that correction back onto PR #24.

27. assistant (2026-07-06T07:02:19.933Z)

Branch and HEAD are exactly where expected, and the diff check is clean. I?m staging only `SESSION_HANDOFF.md` and committing the PR review fix.

28. assistant (2026-07-06T07:02:36.040Z)

The commit completed; that concurrent status snapshot was mid-commit, so I?m taking a fresh status and then rerunning the gates after the doc-only fix for a clean review record.

29. assistant (2026-07-06T07:02:44.806Z)

The PR branch is clean and now one commit ahead of the original PR head; `origin/main..HEAD` contains only the original tracker commit plus my handoff-pointer fix. I?m rerunning the full verification set once more before pushing and approving.

30. assistant (2026-07-06T07:03:00.915Z)

All gates are green after the handoff fix: 364 tests, 95.38% coverage, default ruff clean, and mypy clean once the missing reportlab stubs are installed. I?m doing the final branch check and pushing the PR update now.

31. assistant (2026-07-06T07:03:05.246Z)

HEAD is my handoff-pointer commit and the worktree is clean, ahead by exactly that commit. Pushing it to the PR head branch explicitly.

32. assistant (2026-07-06T07:03:22.530Z)

The PR branch is updated on GitHub. Since the review issue was fixed and the verification gate is now satisfied, I?m going to submit an approving PR review with the exact checks I ran, then merge it so I can start executing the tracker.

33. assistant (2026-07-06T07:03:27.698Z)

GitHub sees the updated PR head. Mergeability is recalculating right after the push, but there are no configured status checks; I?m submitting the approval now with the verification notes.

34. assistant (2026-07-06T07:03:49.708Z)

GitHub rejected the formal approval because the authenticated account is also the PR author. I?ll leave the review result as a PR comment instead, then merge from the verified head.

35. assistant (2026-07-06T07:04:06.505Z)

The review comment is posted and GitHub now reports the PR as mergeable. I verified `origin/main..HEAD` contains only the PR?s tracker commit plus my handoff fix, so I?m merging it now.

36. assistant (2026-07-06T07:04:49.878Z)

PR #24 is merged. I?m going to pick up the new tracker in a fresh implementation worktree from

`origin/main` so the dirty primary checkout?s handoff edits stay untouched. This next phase is broad, so I?ll keep the work organized around the tracker rows and verify hard after each meaningful slice.

37. assistant (2026-07-06T07:04:55.230Z)

`origin/main` now includes PR #24 at merge commit `5cf06c5`. The primary checkout still has an unrelated modified handoff, so I?m branching from the remote into a separate worktree.

38. assistant (2026-07-06T07:05:08.941Z)

The implementation branch is checked out at the merged tracker. Per convention, I?m fast-forward pulling in that new worktree before reading the tracker and code.

39. assistant (2026-07-06T07:05:18.278Z)

The tech-debt worktree is clean and current. I?m reading the merged tracker plus the files it calls out, then I?ll turn it into an implementation checklist.

40. assistant (2026-07-06T07:05:24.236Z)

I?ve got the tracker in view. It?s a lot of surface area, so I?m going to work in dependency order: packaging/tooling/CI first, data-loss and index correctness next, then the baseline split/CLI/test/config cleanup, then archive the tracker.

41. assistant (2026-07-06T07:05:37.127Z)

Starting TD1-TD3 now. I?m reading the CLI, config, tool shim, and tests around command dispatch so the packaging fix doesn?t subtly change how local repo-root discovery works.

42. assistant (2026-07-06T07:05:57.316Z)

I?m making the packaging/tooling slice now: explicit setuptools package list and console script, dev extras that actually include the advertised tools, committed ruff/mypy config, CI, and a repo-root CLI option so installed `agent-archive` can run from the archive checkout without relying on package file location.

43. assistant (2026-07-06T07:06:09.673Z)

The package metadata and CI are in place. Next I?m adjusting the wrapper and CLI root handling so `agent-archive` is the primary entrypoint and `tools/agent_archive.py` no longer mutates `sys.path`.

44. assistant (2026-07-06T07:06:25.716Z)

The CLI no longer has a module-level repo root to monkeypatch, so I?m updating the tests to run `main()` from the temp repo root, which is closer to the installed-console-script behavior anyway.

45. assistant (2026-07-06T07:06:34.891Z)

With dev extras now carrying the tools, I?m tightening README setup instructions so a fresh clone has one supported install command instead of the old failing split install.

46. assistant (2026-07-06T07:06:44.502Z)

I?m checking the packaging slice before moving on: editable install, console script help, and the current Python gates. If this slice is clean, I can build the later refactors on a stable toolchain.

47. assistant (2026-07-06T07:07:17.011Z)

The first install check caught a useful packaging detail: modern setuptools requires a valid

SPDX string if `license` is a plain string, and `Private` is not one. I?m reverting that field to the accepted table form so the real packaging fix can be verified.

48. assistant (2026-07-06T07:07:21.458Z)

Retrying the editable install and console-script smoke test with the corrected license metadata.

49. assistant (2026-07-06T07:08:05.771Z)

TD1?s install path is now green: editable install succeeds and `agent-archive --help` works. I?m running the Python gates before moving into the data-integrity changes.

50. assistant (2026-07-06T07:08:22.594Z)

Packaging/tooling/CI are verified: editable install passes, console script works, 365 tests pass at 95.39%, ruff clean, mypy clean. I?m moving to the data-integrity slice next.

51. assistant (2026-07-06T07:08:35.295Z)

For TD4-TD9/TD15 I?m reading the archive/index/status code and the promotion/ledger helpers together. The goal here is to make the data shape coherent once, then have tests enforce the edge cases that used to be traps.

52. assistant (2026-07-06T07:09:06.607Z)

I?m adding the shared primitives first: tolerant JSONL with warnings, POSIX path normalization, file-state fingerprints, and atomic text replacement. Then archive/status/baseline can use the same small tools instead of each hand-rolling a policy.

53. assistant (2026-07-06T07:09:21.564Z)

Now I?m wiring those primitives into archive export/status: normalized source paths, SHA-based identity with same-path replacement, atomic index writes, missing-source warnings, UTC stems, and a prune command for stale records/files.

54. assistant (2026-07-06T07:10:29.966Z)

Archive export now writes normalized source paths and can reuse unchanged indexed files by size/mtime. I?m updating status to use the same identity logic so multi-machine duplicates and local changed files are classified correctly.

55. assistant (2026-07-06T07:10:47.400Z)

One subtlety in status: a changed local file should count as changed, not as both changed and ?not visible.? I?m adding a visible-source-file set so the digest re-keying doesn?t inflate that number.

56. assistant (2026-07-06T07:10:56.325Z)

Next are the two explicit data-loss traps: ledger writes and promote. I?m making ledger replacement atomic and changing promote to update marker blocks without discarding hand-written prose around them.

57. assistant (2026-07-06T07:11:16.793Z)

The baseline file only imported one utility, so I?m applying the ledger/promote changes against the current import shape rather than the later split target.

58. assistant (2026-07-06T07:11:59.300Z)

For promotion, I?m replacing the ?rebuild the whole file from sorted blocks? behavior with a marker-aware upsert: existing marker blocks are replaced in place, new blocks append at the end, and unmarked prose stays where a human put it.

59. assistant (2026-07-06T07:12:15.119Z)

The prune implementation needs a CLI hook, so I?m adding a top-level `prune` command now; the broader CLI registration refactor will build on this in the next slice.

60. assistant (2026-07-06T07:12:27.597Z)

I?m handling the ?silent failures? row next: config gets explicit typed errors for malformed source entries, and WSL discovery becomes cached so repeated path-template resolution doesn?t shell out over and over.

61. assistant (2026-07-06T07:12:33.838Z)

The repo requires Python 3.11+, so I?m also dropping the obsolete `tomllib` fallback while I?m touching config; malformed source entries will now raise a named config error instead of a raw `KeyError`.

62. assistant (2026-07-06T07:13:07.866Z)

A small status accounting polish while I?m here: identity keys can dedupe records by SHA, but ?visible files? should still count actual configured files. I?m separating those two counters.

63. assistant (2026-07-06T07:13:15.914Z)

The data-integrity code is in. I?m running the archive/status/baseline/config/path-template tests now to catch the obvious behavior shifts before I add the new regression tests.

64. assistant (2026-07-06T07:13:26.133Z)

The expected failures are from tests that still describe the old behavior: the Python <3.11 TOML fallback and `SystemExit` for unknown templates. I?m updating those tests to the new typed-error behavior and adding coverage for malformed source config.

65. assistant (2026-07-06T07:13:45.453Z)

Now I?m adding the regression tests that match the tracker rows: SHA de-duping, UTC date stems, prune behavior, POSIX index paths, CLI prune parsing, and manual prose preservation in promote.

66. assistant (2026-07-06T07:14:16.019Z)

The regression tests are in place. I?m rerunning the focused suite to shake out behavior or formatting issues in this slice.

67. assistant (2026-07-06T07:14:24.423Z)

Focused tests pass. I?m going to make that unknown-source warning explicit in its test so the suite doesn?t carry a stray warning, then I?ll do a lint/type pass before the larger baseline-module split.

68. assistant (2026-07-06T07:14:36.249Z)

Running ruff and mypy over the data-integrity slice now; this should catch import ordering, long lines, and type edge cases before I start moving baseline code around.

69. assistant (2026-07-06T07:14:49.280Z)

Mypy found two narrow optional-typing issues from the new reuse/status logic. I?m tightening those branches so the type checker can see the same guarantees the code has at runtime.

70. assistant (2026-07-06T07:15:05.358Z)

Those typing fixes are in; rerunning mypy quickly before moving on.

71. assistant (2026-07-06T07:15:13.058Z)

The data-integrity slice is implemented and type-clean: promote preserves manual text, ledger/index writes are atomic, archive stems use UTC, records are SHA-keyed with normalized source paths, prune exists, JSONL parsing is tolerant with warnings, and status/export can short-circuit unchanged files. Next is the baseline split and CLI cleanup.

72. assistant (2026-07-06T07:15:32.767Z)

For TD10 I?m mapping `baseline.py` into natural chunks before moving code: types/settings, prediction generation, reports/artifacts, promotion, and orchestration. I want the facade file to get small without creating a tangled import graph.

73. assistant (2026-07-06T07:16:15.637Z)

For the baseline split, I?m going to turn `agent_sessions.baseline` into a small compatibility facade and move the implementation behind narrower modules. That preserves public imports for tests/users while breaking the top-level calibration cycle and getting the facade well under the tracker?s size target.

74. assistant (2026-07-06T07:16:35.645Z)

The old implementation is now `baseline_core.py`. I?m adding the public facade plus the narrower import modules, then I?ll update internal imports to use those narrower modules instead of reaching through the facade.

75. user (2026-07-06T07:17:08.952Z)

Please make sure to send individual PRs for review as you work through this project and wait for the PR comments to address them. Once there is an LGTM on the PR, then you can finalize and merge the PRs andkeep going.

76. user (2026-07-06T07:17:08.972Z)

and are you not sending isolated PRs? or are sending a single PR to fix them all?

77. assistant (2026-07-06T07:17:24.010Z)

You?re right to call that out. I had started implementing on one broad local branch after PR #24, but I have not pushed or opened an implementation PR yet; I?ll pivot now to isolated PRs and wait for review/LGTM on each one before merging and continuing.

78. assistant (2026-07-06T07:17:25.488Z)

I?m going to split the work already started instead of shipping it as one blob. First PR will be TD1-TD3 only: packaging, dev tool config, and CI. I?ll keep the later data-integrity/baseline changes out of that PR.

79. assistant (2026-07-06T07:17:36.765Z)

I?ve loaded the publish workflow. Because the current worktree is mixed, I?m inspecting the diff and then I?ll either carve out TD1-TD3 into a clean branch or create a fresh worktree from `origin/main` and reapply only that slice.

80. assistant (2026-07-06T07:17:50.126Z)

The current branch has mixed TD1-TD15 edits, including generated `agent_sessions.egg-info`, so I?m not going to stage from it. I?m creating a fresh `codex/tech-debt-td1-tooling` worktree from `origin/main` and reapplying only the TD1-TD3 tooling slice there.

81. assistant (2026-07-06T07:18:05.629Z)

One extra detail: that PR head branch moved again after merge, so I?m staying pinned to

`origin/main` for the implementation PRs. Creating the isolated TD1-TD3 worktree now.

82. assistant (2026-07-06T07:18:26.487Z)

The clean TD1-TD3 branch is checked out. I?m doing its fast-forward pull, then I?ll apply just the packaging/tooling changes.

83. assistant (2026-07-06T07:18:32.525Z)

Applying the isolated TD1-TD3 changes now: packaging metadata, console script, dev extras/tool config, CI workflow, repo-root CLI option, wrapper cleanup, and matching README/tests.

84. assistant (2026-07-06T07:18:45.146Z)

Next I?m applying the CLI/wrapper bits for the same PR. This is still TD1 scope: make the installed console command usable from a repo checkout, and remove the wrapper?s path mutation.

85. assistant (2026-07-06T07:18:57.137Z)

I?m updating the CLI tests to use `monkeypatch.chdir()` instead of patching a removed module global, and adding a parser assertion for `--repo-root`.

86. assistant (2026-07-06T07:19:06.988Z)

Finishing the TD1-TD3 doc changes now: README will advertise the now-working single dev install command and CI-backed checks.

87. assistant (2026-07-06T07:19:16.324Z)

The first isolated PR scope is now applied. I?m verifying from a clean venv: editable install, console script, full tests with coverage, ruff, and mypy.

88. assistant (2026-07-06T07:20:14.405Z)

Install and `agent-archive --help` are good for the isolated branch. The install created local egg-info metadata, so I?ll ignore/remove that before staging after the rest of the checks.

89. assistant (2026-07-06T07:20:32.688Z)

All TD1-TD3 checks are green: install, console script, full suite with coverage, ruff, mypy. I?m cleaning the generated egg-info from the worktree and then I?ll commit only the intended files.

90. assistant (2026-07-06T07:20:37.805Z)

The only untracked non-source item is the generated `agent_sessions.egg-info` from the editable install. I?m removing that generated directory explicitly, then doing the branch/HEAD verification required before staging.

91. assistant (2026-07-06T07:20:57.339Z)

The TD1-TD3 worktree is clean apart from the intended files. I?m staging only those explicit paths and committing the isolated PR.

92. assistant (2026-07-06T07:21:18.293Z)

The TD1-TD3 commit is made on the isolated branch, and `gh` is authenticated. I?m doing the pre-push verification now: branch, HEAD, clean worktree, and only this commit over `origin/main`.

93. assistant (2026-07-06T07:21:24.655Z)

The branch is clean, ahead by exactly the TD1-TD3 commit. Pushing and opening the review PR now.

94. assistant (2026-07-06T07:21:33.109Z)

Branch is pushed. I?m creating a ready-for-review PR rather than a draft, since your workflow needs comments/LGTM before I merge and continue.

95. assistant (2026-07-06T07:22:00.159Z)

You?re right: I had started too broadly locally, but I did not push a single omnibus implementation PR.

I corrected course and opened the first isolated review PR:

[PR #28: tech-debt: add packaging and CI tooling](https://github.com/avidullu/agent-sessions/pull/28)

Scope is only TD1-TD3: packaging, dev tooling config, and CI. Validation passed locally: editable install, `agent-archive --help`, full pytest with 95.39% coverage, ruff, and mypy.

I?ll wait for PR comments / LGTM before merging #28 or starting the next tracker slice.

::git-stage{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-td1-tooling"}
::git-commit{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-td1-tooling"}
::git-create-branch{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-td1-tooling" branch="codex/tech-debt-td1-tooling"}
::git-push{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-td1-tooling" branch="codex/tech-debt-td1-tooling"}
::git-create-pr{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-td1-tooling" branch="codex/tech-debt-td1-tooling" url="https://github.com/avidullu/agent-sessions/pull/28" isDraft=false}

96. user (2026-07-06T07:22:00.853Z)

Can you make this as a default convention to alsways operate in PR - Review - Resolve - Merge methodology and only do this sledge hammer approach when doing a time critical or convoluted/hairy change? I want other sessions and maybe agents to follow this as well

97. assistant (2026-07-06T07:22:17.218Z)

Yes. I?ll codify that as repo guidance rather than just a preference in this chat, and I?ll do it through the PR workflow we just agreed on: isolated branch, small doc change, PR for review. First I?m refreshing the checkout and locating the right instruction file.

98. assistant (2026-07-06T07:22:31.165Z)

The primary checkout can?t fast-forward because `SESSION_HANDOFF.md` has local WIP, so I?m leaving it untouched and using a fresh worktree from `origin/main`. There?s no repo `AGENTS.md` yet, so I?m going to add one at the repo root with this convention, which is the most direct way to make future sessions/agents see it.

99. assistant (2026-07-06T07:23:02.844Z)

The convention will live in a new root `AGENTS.md`, since that filename is what many coding agents naturally look for. I?m making it explicit that normal work is small PRs with review resolution and LGTM before merge, with the broad ?sledgehammer? branch only allowed as an exception for emergencies or genuinely hairy exploration.

100. assistant (2026-07-06T07:23:07.508Z)

The convention worktree is clean and current. I?m adding the instruction file now.

101. assistant (2026-07-06T07:23:27.090Z)

The new `AGENTS.md` is intentionally short and repo-level. I?m verifying the diff, then I?ll commit and open a PR for this convention change as requested.

102. assistant (2026-07-06T07:23:40.192Z)

`git diff --check` is clean; the file is new and untracked, so I?m reading it once before staging, then committing just that file.

103. assistant (2026-07-06T07:23:50.258Z)

The convention is committed on its isolated branch. I?m doing the pre-push branch/HEAD check, then I?ll push and open the PR.

104. assistant (2026-07-06T07:23:54.120Z)

The branch is clean and ahead by only the convention commit. Pushing now and opening it for review.

105. assistant (2026-07-06T07:24:01.995Z)

Branch is pushed. I?m creating a ready-for-review PR and then I?ll stop for review/LGTM, per the new convention.

106. assistant (2026-07-06T07:24:23.243Z)

GitHub timed out while creating the PR, so I?m checking whether it actually created one before retrying.

107. assistant (2026-07-06T07:24:29.245Z)

No PR was created, so I?m retrying the PR creation.

108. assistant (2026-07-06T07:24:45.491Z)

Done. I added a root `AGENTS.md` with the default convention:

PR -> review -> resolve comments -> LGTM/explicit approval -> merge -> refresh from remote -> next slice.

It also names the exception: broad/sledgehammer work is only for time-critical or genuinely hairy exploration, and should be split back into reviewable PRs before shipping when practical.

Opened for review here: [PR #29](https://github.com/avidullu/agent-sessions/pull/29)

Validation: `git diff --check`. I?ll wait for comments/LGTM before merging it.

::git-stage{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-pr-convention"}
::git-commit{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-pr-convention"}
::git-create-branch{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-pr-convention" branch="codex/pr-review-merge-convention"}
::git-push{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-pr-convention" branch="codex/pr-review-merge-convention"}
::git-create-pr{cwd="C:\Users\avidu\Documents\Codex\2026-07-06\agent-sessions-pr-convention" branch="codex/pr-review-merge-convention" url="https://github.com/avidullu/agent-sessions/pull/29" isDraft=false}

109. user (2026-07-06T07:25:35.839Z)

LGTM'd.

110. assistant (2026-07-06T07:25:56.692Z)

Great. I?ll verify which PR has the LGTM, merge the approved one, and then refresh the branch state before doing anything else.

111. assistant (2026-07-06T07:26:06.896Z)

PR #29 has the owner LGTM/comment and is mergeable. PR #28 is still open with CI green but no LGTM/comments, so I?m merging only #29.

112. assistant (2026-07-06T07:26:19.032Z)

The convention PR is merged. I?m fetching the remote state now so the next work starts from the updated base.

113. assistant (2026-07-06T07:26:26.267Z)

Merged PR #29: [docs: add PR review workflow convention](https://github.com/avidullu/agent-sessions/pull/29). It landed as merge commit `d13beb8`.

I also checked PR #28: CI is green, but there are no LGTM/comments on it yet, so I?m leaving it open and unmerged.

114. user (2026-07-06T07:36:24.011Z)

can you fix the comments in #28 and move ahead with the project work

115. assistant (2026-07-06T07:36:37.679Z)

I?ll inspect PR #28?s comments/review threads, make the smallest fixes on that PR branch, push the update, and then follow the new rule: merge only if the comments include an explicit LGTM/approval after the fixes. After that I?ll continue with the next isolated tracker PR from updated `origin/main`.

116. assistant (2026-07-06T07:36:55.100Z)

I?m using the `gh-address-comments` workflow here because this is specifically about unresolved PR feedback. First I?m refreshing the #28 worktree and reading both top-level comments and inline review threads.

117. assistant (2026-07-06T07:37:05.901Z)

PR #28 has no inline review threads. There are two owner comments: an LGTM/clear-to-merge, plus a small packaging follow-up to ignore generated Python build artifacts. I?m folding that `.gitignore` fix into #28, rerunning checks, then I can merge #28 because the LGTM explicitly still stands.

118. assistant (2026-07-06T07:37:13.657Z)

The repo already ignores `__pycache__/`, so I?m adding the missing packaging artifact patterns only: `*.egg-info/`, `build/`, and `dist/`.

119. assistant (2026-07-06T07:37:22.837Z)

I?m verifying the exact reported issue now by rerunning editable install and checking that the generated egg-info no longer shows up as untracked, then I?ll rerun the PR gates.

120. assistant (2026-07-06T07:37:44.160Z)

The install reproduced `agent_sessions.egg-info/`, and it is now ignored instead of untracked. I?m rerunning tests, ruff, and mypy after the comment fix.

121. assistant (2026-07-06T07:38:53.388Z)

I?m picking back up in the isolated #28 worktree, first re-running the local gates so the comment fix is verified before I push or merge anything. After that I?ll merge #28 since it already has LGTM, then start the next tracker item as a separate PR.

122. assistant (2026-07-06T07:39:15.532Z)

The full pytest run is clean again: 365 passed with 95.39% coverage, comfortably over the repo?s 80% gate. I?m checking lint and typing next, then I?ll commit just the `.gitignore` follow-up.

123. assistant (2026-07-06T07:39:23.632Z)

Lint and mypy are clean too, and the diff is exactly the ignore-pattern update requested in the PR comment. I?m doing the branch/HEAD safety check now, then staging only `.gitignore`.

124. assistant (2026-07-06T07:39:34.067Z)

The follow-up is committed as a single-file commit on the #28 branch. I?m pushing it to the existing PR now, then I?ll confirm the PR state before merging.

125. assistant (2026-07-06T07:39:45.110Z)

The PR is still mergeable and the LGTM is intact; the branch now includes the requested ignore-pattern commit. I?m going to wait for the pushed commit?s GitHub check to settle before merging, even though the local full suite is already green.

126. assistant (2026-07-06T07:40:33.923Z)

GitHub CI has passed on the updated #28 commit. I?m merging #28 now, then I?ll refresh from `origin/main` and start the next tracker task on a new branch.

127. assistant (2026-07-06T07:40:52.218Z)

#28 is merged: merge commit `8ae8eb3`. I?m fetching the updated remote and reading the project tracker from a fresh branch point so the next PR starts from the actual current `main`, not from the older tooling branch.

128. assistant (2026-07-06T07:41:02.026Z)

The primary checkout is still dirty and behind, so I?m leaving it alone and using a clean sibling worktree for the next isolated PR. That keeps the local handoff WIP untouched while still branching from `origin/main`.

129. assistant (2026-07-06T07:41:21.010Z)

The new TD4 branch is clean and based on the merged #28 state. I?m reading the handoff and tracker files now so I can keep the next PR to one project slice.

130. user (2026-07-06T07:41:33.170Z)

<turn_aborted>
The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.
</turn_aborted>